COP 5536 Fall 2026

Programming Project
Due – November 9th, 2026

Job Scheduler

Problem Description

Schedulers are used whenever a computing system has more work available than it can execute at the same time. An operating system scheduler decides which runnable process or thread should receive CPU time. In high-performance computing (HPC), a batch scheduler performs a similar role at a larger scale: jobs are submitted to a queue and are started when processors, memory, accelerators, or other requested resources become available.

For example, Linux uses a CPU scheduler to share processor time among runnable tasks. HPC clusters commonly use workload managers such as Slurm or PBS/OpenPBS to accept batch jobs, maintain waiting queues, apply scheduling policies, and launch jobs when resources become available. These production systems are much more complicated than what is required in this project, but they provide the motivation for the simplified scheduler described below.

In this project, you will build a small job-management and scheduling simulator. The program will not execute real applications. Instead, it will read job requests and scheduler commands from a text file and simulate when jobs enter the system, when they start, and when they complete.

System Overview

The system has a fixed execution capacity, denoted by k. At most k jobs may be RUNNING at the same time. Jobs that have been submitted but not yet started remain in a waiting queue. The system begins in PRIORITY scheduling mode, where the waiting job with the highest priority is selected first. The input may later contain a SHORT_JOBS command. Once this command is encountered, the scheduling mode changes permanently for the remainder of the input file and the waiting job with the shortest estimated execution time is selected first.

Each job contains the following information:

- Job ID - a unique integer identifier.
- Submission Time - the timestamp, in milliseconds, at which the SUBMIT command appears.
- Priority - an integer scheduling priority. A larger value represents a higher priority.
- Estimated Time - the estimated execution time of the job, in milliseconds.
- Delta - a value supplied in the input file and added to the estimated time to determine the simulated actual execution time.
- Status - WAITING, RUNNING, COMPLETED, or CANCELLED.
- Students must create a Job class (or equivalent record/object type) containing all job information listed above. Each submitted job is represented by one Job object, and references to that same object are stored in the scheduler data structures as described below.

The actual execution time of a job is determined as follows:

```
actual_execution_time = estimated_time + delta
```

The test inputs will guarantee that the resulting actual execution time is greater than zero. Delta values are provided in the input file so that the same input always produces the same result on every implementation.

Data Structures Used in the Project

You are required to implement two data structures from scratch: an AVL Tree and a Binary Max Heap. Standard-library implementations may be used for the List that stores cancelled Job objects and for the priority queue that tracks running jobs.

The AVL Tree is the primary storage and lookup structure for submitted jobs. The Job ID is the key used to maintain the AVL Tree structure, and each AVL Tree node stores a reference to the corresponding Job object. A submitted job remains in the AVL Tree while it is WAITING, RUNNING, or COMPLETED. If a WAITING job is cancelled, it is deleted from the AVL Tree and its Job object is appended to the Cancelled Jobs List.

The Binary Max Heap stores only WAITING jobs and contains references to the same Job objects stored in the AVL Tree. In the default PRIORITY mode, a job with a larger priority has higher scheduling rank. If two jobs have the same priority, the job submitted earlier has higher rank; if both values are the same, the smaller Job ID has higher rank.

After SHORT_JOBS is encountered, the existing waiting heap must be reorganized so that a smaller estimated time has higher scheduling rank. If estimated times are equal, use higher priority, then earlier submission time, then smaller Job ID as tie breakers. The system remains in this mode for the rest of the input file.

The system will maintain a Cancelled Jobs List containing references to Job objects that have been cancelled. Students may use the programming language's standard library List, ArrayList, vector, or equivalent structure for this purpose. When a WAITING job is cancelled, it is removed from the AVL Tree and the waiting Max Heap, and its Job object is appended to this list. Cancelled jobs remain in the list in cancellation order and are never removed.

The system must also keep track of jobs that are currently running. Since the system can execute at most k jobs at a time, it must be able to determine when the next execution slot becomes available. You may use the programming language's standard library **Min-Heap / Priority Queue** for this purpose. Each running job is stored in this structure using its **completion time** as the ordering value. The completion time of a job is calculated as:

The running-job priority queue may also store references to the same Job objects. This avoids creating duplicate job records in different scheduler structures.

```
completion time = execution start time + estimated time + delta
```

Therefore, the running job with the earliest completion time will always be available at the top of the Min-Heap. When a job completes, it is removed from this structure and an execution slot becomes available. Y**ou may use the library implementation of Min-Heap to track execution slot availability.**

AVL Tree Convention

For this project, the AVL Tree is a binary search tree ordered by Job ID. Each node stores the Job ID as its key and a reference to the corresponding Job object. For every node, the height difference between the left and right subtrees must not exceed one.

The balance factor of a node is defined as:

$$BF = height(left) - height(right)$$

Therefore, every node in a valid AVL Tree must have a balance factor of -1, 0, or 1. The height of a leaf node is 1, and the height of an empty subtree is 0.

After every insertion or deletion, the AVL Tree must be checked for imbalance. If a node becomes unbalanced, the appropriate AVL rotation must be performed. AVL rotations and rebalancing must follow the procedure discussed in class.

Simulation Time and Capacity

The first line of every input file specifies system capacity:

```
CAPACITY <k>
```

Every remaining line begins with a timestamp in milliseconds. Commands in the input file are ordered by nondecreasing timestamp. Before processing a command at time t, any RUNNING job whose completion time is less than or equal to t must first be marked COMPLETED. If a completion and an input command occur at the same timestamp, the completion is processed first.

PROCESS_NEXT starts exactly one waiting job. If fewer than k jobs are running, the selected job starts at the command timestamp. If all k slots are occupied, PROCESS_NEXT waits until the earliest running job completes, advances the simulation clock to that completion time, processes every job completing at that same time, and then starts one waiting job.

Supported Operations

1. SUBMIT <job_id> <priority> <estimated_time> <delta>: Submit a new job at the timestamp specified at the beginning of the input line. The job is initially WAITING. Create a Job object containing the complete job information, insert that object into the AVL Tree using Job ID as the key, and insert a reference to the same Job object into the waiting Max Heap. Job IDs are unique.
2. PROCESS_NEXT: Start the next WAITING job according to the current scheduling mode. If the system is at capacity, wait until at least one slot becomes available. If no jobs are waiting, no job is started.
3. FIND <job_id>: Search the AVL Tree using Job ID and print the job's current information: Job ID, Submission Time, Execution Start Time, Actual Execution Time, Completion Time, and Status. If the Job ID is not found in the AVL Tree, search the Cancelled Jobs List. This allows FIND to return information for cancelled jobs as well as jobs that are WAITING, RUNNING, or COMPLETED. Fields that are not yet known are printed as null.
4. JOB_STATUS: Print the current status of all submitted jobs in table form. First perform an inorder traversal of the AVL Tree and print the Job objects stored in the tree; this prints all non-cancelled jobs in increasing Job ID order. Then traverse the Cancelled Jobs List from beginning to end and print the cancelled jobs in cancellation order.

5. CANCEL <job_id>: Cancel a job only if it is WAITING. A RUNNING, COMPLETED, or already CANCELLED job cannot be cancelled. Locate the job by searching the AVL Tree using Job ID. When a WAITING job is cancelled, remove its reference from the waiting Max Heap, delete the Job ID entry from the AVL Tree, update the Job object status to CANCELLED, and append the Job object to the Cancelled Jobs List. FIND and JOB_STATUS must continue to report the cancelled job by searching or traversing this list as described above.
6. UPDATE_PRIORITY <job_id> <new_priority>: Locate the job by searching the AVL Tree using Job ID. Change the priority only if the job is WAITING. A RUNNING or COMPLETED job cannot be updated; a cancelled job will not be present in the AVL Tree. The waiting heap must reflect the new scheduling rank.
7. SHORT_JOBS: Change the scheduler to shortest-job mode for the remainder of the input file. The existing waiting heap must be reorganized using estimated time as the primary scheduling value. All later submitted jobs use the same mode.

Programming Environment

You may use Java, C++, or Python for this project. Your program will be compiled and tested on an Ubuntu machine running the latest Ubuntu LTS release with the default Java, Python 3, GCC, and G++ installations available on that system.

Your submission must include a Makefile. Running make from the top-level submission directory must prepare the program for execution. If your solution depends on any library that is not part of the default language installation, the dependency must be installed or made available by the Makefile without administrative privileges and without interactive input. Do not assume that additional packages are preinstalled on the grading machine.

The program should execute using one of the following forms:

```
C/C++:  ./jobScheduler input_file
Java:   java jobScheduler input_file
Python: python3 jobScheduler.py input_file
```

The input file name must be supplied as a command-line argument.

Input and Output Requirements

- The first non-empty line of the input file will be CAPACITY <k>.
- Each subsequent command begins with a nonnegative integer timestamp measured in milliseconds.
- Input timestamps appear in nondecreasing order.
- If a `PROCESS_NEXT` command must wait because all execution slots are occupied, the simulation time may advance to the completion time of the earliest running job. The test files will be constructed such that no later input command has a timestamp earlier than the simulation reached by this wait. Therefore, the simulator will never be required to move backward in time.
- All program output must be written to a text file named <input-base>_output_file.txt. For example, test1.txt produces test1_output_file.txt.
- Output must be deterministic. Do not print memory addresses, object IDs, debugging messages, or graphical tree drawings.
- At the end of the input file, print the execution order, the AVL Tree, the AVL Tree inorder traversal, the Max Heap, and the required statistics.

Function Output Format

The exact wording below is used so that outputs can be graded automatically. Whitespace between table columns may vary, but labels, field order, and values must follow the specified format.

SUBMIT

```
Input:  <time> SUBMIT <job_id> <priority> <estimated_time> <delta>
Output: Job <job_id> submitted at <time> ms
```

If the Job ID already exists:

```
ERROR: Job <job_id> already exists
```

PROCESS_NEXT

```
Job <job_id> started at <start_time> ms
```

If no jobs are waiting:

```
NO_WAITING_JOBS
```

FIND

```
Job <job_id>: submitted=<submission_time>, started=<start_time|null>,
actual_time=<actual_time|null>, completed=<completion_time|null>, status=<status>
```

If the Job ID does not exist:

```
ERROR: Job <job_id> not found
```

JOB_STATUS

```
JOB STATUS
Job     Submitted   Started   Actual Time   Completed   Status
<id>    <time>      <time>    <time>        <time>      <status>
...
END JOB STATUS
```

Use null for Started, Actual Time, or Completed when the corresponding value is not yet available.

CANCEL

```
Job <job_id> cancelled
```

If the job cannot be cancelled:

```
ERROR: Job <job_id> cannot be cancelled while status is <status>
```

If the Job ID does not exist:

```
ERROR: Job <job_id> not found
```

UPDATE_PRIORITY

```
Job <job_id> priority updated to <new_priority>
```

If the priority cannot be updated:

```
ERROR: Job <job_id> priority cannot be updated while status is <status>
```

If the Job ID does not exist:

```
ERROR: Job <job_id> not found
```

SHORT_JOBS

```
Scheduling mode changed to SHORT_JOBS
```

Required Final Structural Output

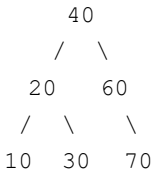
At the end of every input file, your program must print the internal state of the two data structures that you implemented. These representations are part of the graded output. They must be generated directly from the actual data structures used by your program.

AVL Tree Output

Print the AVL Tree level by level, from the root downward. Nodes on the same level are separated by the character |. Scheduler keys are Job IDs and are printed as integers. Use NULL for a missing child when needed to preserve the shape of a level. Trailing NULL entries after the last real node on a level do not need to be printed.

```
TREE
LEVEL 0: [<key>]
LEVEL 1: [<key>]|[<key>]
LEVEL 2: [<key>]|NULL|[<key>]|[<key>]
...
END_TREE
```

The following small example uses integer keys (Job ID) to demonstrate the required output format.



The required text output for this tree is:

```
TREE
LEVEL 0: [40]
LEVEL 1: [20]|[60]
LEVEL 2: [10]|[30]|NULL|[70]
END_TREE
```

Inorder Output

Print an inorder traversal of the AVL Tree.

```
INORDER: [10, 20, 30, 40, 60, 70]
```

Max Heap Output

Print the Max Heap in its actual internal array order. Each scheduler heap entry is printed as (job_id,priority,estimated_time).

```
MAX_HEAP: [(<job_id>,<priority>,<estimated_time>), ...]
```

The following example uses simple integer values only to demonstrate the output format. It is not a scheduler heap.

```
             90
           /    \
         70      80
        /  \    /
      20   60  50

MAX_HEAP: [90,70,80,20,60,50]
```

Execution Order

At the end of the input file, print jobs in the order in which they started execution. Jobs that never started are not included in this section.

```
EXECUTION_ORDER
<job_id> <start_time> <completion_time>
...
END_EXECUTION_ORDER
```

Statistics Output

The following statistics are also required. The simple values below are only an illustration; they are not tied to the scheduler examples.

```
TREE_STATS
records=6
height=3
insertions=7
deletions=1
left_rotations=1
right_rotations=1
END_TREE_STATS

HEAP_STATS
size=6
max_size=6
insertions=6
extractions=0
removals=0
priority_updates=0
rebuilds=0
END_HEAP_STATS
```

For the AVL Tree, height is measured using the convention given above: an empty subtree has height 0 and a leaf has height 1. The rotation counts report the number of primitive left and right rotations performed by the AVL Tree implementation. For the heap, “rebuilds” counts the number of times the waiting heap is reorganized because of SHORT_JOBS.

Examples

The examples below focus on scheduler behavior. To keep them readable, the final structural output is shown in abbreviated form where appropriate; the program must still produce the complete final AVL Tree, inorder traversal, Max Heap, execution order, and statistics described above.

Example 1 - Simple Priority Scheduling

This example uses one execution slot. Job 102 has higher priority and is selected first even though it was submitted later.

Input

```
CAPACITY 1
0 SUBMIT 101 5 100 10
5 SUBMIT 102 8 50 5
10 PROCESS_NEXT
20 FIND 102
65 PROCESS_NEXT
175 JOB_STATUS
```

Output (functional portion)

```
Job 101 submitted at 0 ms
Job 102 submitted at 5 ms
Job 102 started at 10 ms
Job 102: submitted=5, started=10, actual_time=55, completed=65, status=RUNNING
Job 101 started at 65 ms
JOB STATUS
Job     Submitted   Started   Actual Time   Completed   Status
101     0           65        110           175         COMPLETED
102     5           10        55            65          COMPLETED
END JOB STATUS
EXECUTION_ORDER
102 10 65
101 65 175
END_EXECUTION_ORDER
```

Example 2 - Short-Job Scheduling

This example changes the scheduling policy before any job starts. Job 202 has the lowest priority of the three jobs, but it has the shortest estimated time and is therefore selected first. Notice that Job 202 and Job 201 both have an actual execution time of 110 ms; SHORT_JOBS uses the estimated time only and does not add Delta when ordering waiting jobs.

Input

```
CAPACITY 2
0 SUBMIT 201 10 100 10
1 SUBMIT 202 3 10 100
2 SUBMIT 203 8 50 5
3 SHORT_JOBS
4 PROCESS_NEXT
5 PROCESS_NEXT
```

```
6 PROCESS_NEXT
170 JOB_STATUS
```

Output (functional portion)

```
Job 201 submitted at 0 ms
Job 202 submitted at 1 ms
Job 203 submitted at 2 ms
Scheduling mode changed to SHORT_JOBS
Job 202 started at 4 ms
Job 203 started at 5 ms
Job 201 started at 60 ms
JOB STATUS
Job     Submitted   Started   Actual Time   Completed   Status
201     0           60        110           170         COMPLETED
202     1           4         110           114         COMPLETED
203     2           5         55            60          COMPLETED
END JOB STATUS
EXECUTION_ORDER
202 4 114
203 5 60
201 60 170
END_EXECUTION_ORDER
```

Submission Requirements

Your submission must be a single ZIP file. Do not use nested directories. All source files, the Makefile, and any required dependency/configuration files must appear in the first directory after the ZIP file is extracted.

1. Makefile: running make must prepare the program for execution on the grading Ubuntu system. If additional libraries are required, the Makefile must make them available without requiring administrative access or interactive input.

2. Source Program: submit all source files. The AVL Tree and Binary Max Heap must be your own implementations. You may not replace either with built-in implementations.

The submission must include a Job class (or equivalent type) used to represent each job object stored by the scheduler.

3. Program Structure: keep the AVL Tree and Max Heap implementations separate from the scheduler/input parsing logic. The instructor may inspect or test these implementations independently.

4. Report: submit a short PDF report describing your program organization, the classes/modules used, how the two required data structures are represented, and any additional libraries used.

Your program will be graded primarily using automated test inputs and the produced output. Correct formatting is therefore part of the project requirements. A program that produces correct scheduler messages but an incorrect AVL Tree, inorder traversal, or Max Heap representation will not receive full credit for data-structure correctness.